

PRALLO – Théo Minger

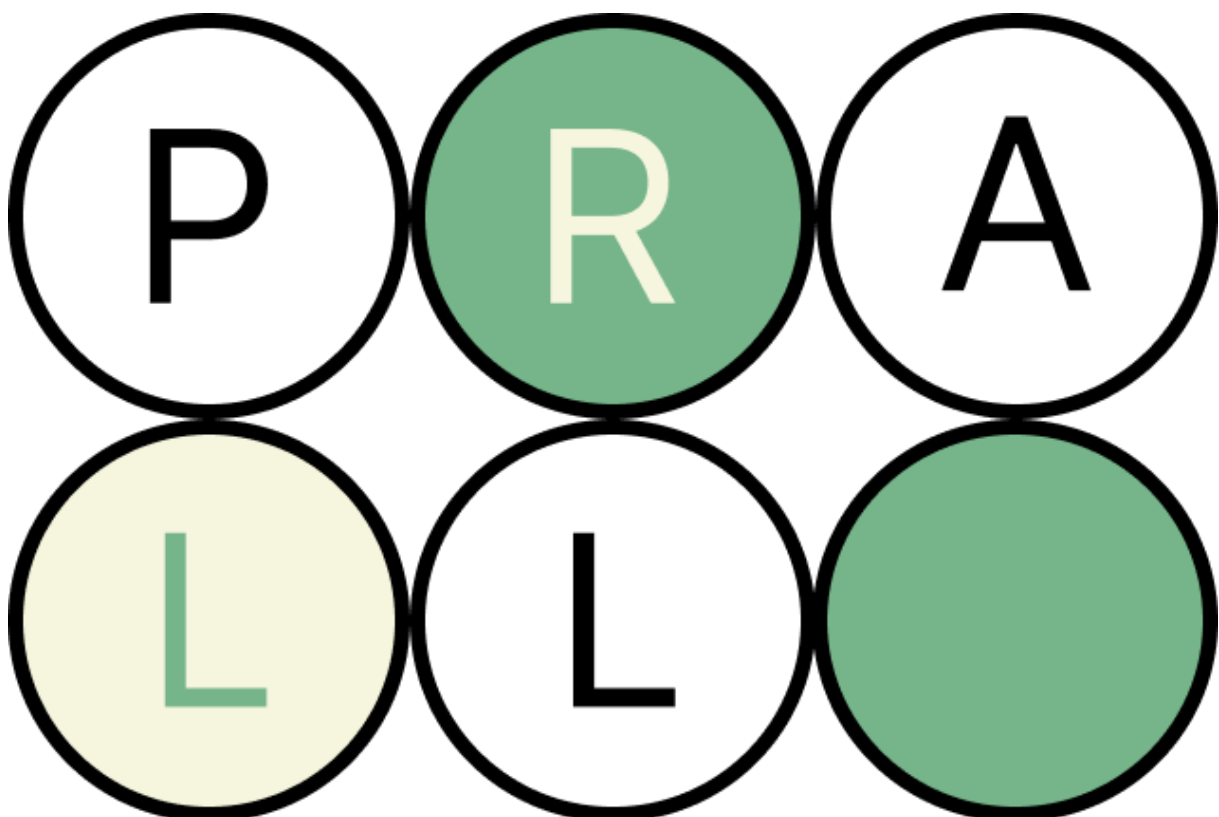


Table des matières

1	Analyse préliminaire	4
1.1	Introduction	4
1.1.1	Résumé	4
1.1.2	Prérequis	4
1.2	Objectifs.....	4
1.2.1	Objectifs de formation.....	4
1.2.2	Objectifs du produit.....	5
1.3	Gestion de projet	5
1.3.1	Outils	5
1.3.2	Pratiques	5
1.3.3	Project-Handbook.....	6
1.3.4	Personnes impliquées et rôles.....	7
1.4	Planification initiale	7
2	Analyse / Conception.....	8
2.1	Lexique	8
2.2	Contexte produit	9
2.3	Contextes techniques	9
2.3.1	Opérationnel	9
2.3.2	Staging	9
2.3.3	Développement.....	10
2.4	L'architecture et les services en schéma	10
2.4.1	Schéma de l'environnement de développement.....	10
2.4.2	Schéma de l'environnement de staging.....	11
2.4.3	Digramme d'état	13
2.4.4	Diagramme de flux concernant l'authentification	15
2.5	Analyse fonctionnelle.....	16
2.5.1	Discussion avec le Client/Product Owner	16
2.5.2	User story	17
2.6	Stratégie de test.....	18
2.7	Risques techniques	19
2.8	Planification	19
3	Réalisation.....	20
3.1	Déroulement.....	20

3.1.1	Points de situation (« sprints »).....	20
3.1.2	Stories	22
3.2	Description des tests effectués	22
3.3	Bilan.....	22
3.3.1	Erreurs restantes	22
3.3.2	Stories	22
3.3.3	Dette technique.....	22
3.4	Recours à l'intelligence artificielle	22
3.5	Liste des documents fournis	22
4	Conclusions	22
5	Annexes.....	23
5.1	Résumé du rapport du TPI / version succincte de la documentation	23
5.2	Sources – Bibliographie	23
5.3	Journal de travail	23
5.4	Manuel d'Utilisation.....	23

1 Analyse préliminaire

1.1 Introduction

1.1.1 Résumé

Chaque année, lors de leur 4ème année, les élèves en informatique de l'ETML se voient attribuer un projet « P_APPRO ». En effet, avant de commencer le TPI, ceux-ci doivent se réhabituer aux pratiques utiliser sur place. Pour se faire, chaque étudiant reçoit un projet à réaliser et un professeur référant. Mais, comment cela se passe-t-il ? En effet, l'attribution d'un projet et d'un professeur par élèves est parfois chaotique.

Cela dépend de beaucoup de chose : des envies de l'élèves, des facultés enseignées par les professeurs, du nombre d'élèves par professeur et j'en passe ...

C'est ici que **Prallo** entre en jeu ! **Prallo** est une application web qui permet de rendre ce moment chaotique un peu plus simple.

1.1.2 Prérequis

Voici une liste des cours réalisé pour permettre la bonne réalisation du projet :

• C294 – Réaliser le front-end d'une application Web interactive
• C295 – Réaliser le back-end pour des applications
• P_WEB – projet Web
• P_BULLE – Snake
• I162 – Analyser et modéliser des données
• I164 – Créer des bases de données
• I165 – Utiliser des bases de données NoSQL
• C106 – Interroger, traiter et assurer la maintenance des bases de données
• P_DB – Projet base de données

1.2 Objectifs

1.2.1 Objectifs de formation

L'intention de ce projet est le suivant : Réaliser un UX/UI user friendly pour une application web complexe qui pourrait être utilisé par n'importe quelle école/entreprise, pour ensuite la pousser en production. Ce que ce projet m'apporte :

- **Gestion de projet** : je suis capable de gérer mon propre projet, en utilisant des méthodes agiles. J'utilise des user stories pour clarifier les objectifs. Je manipule Github pour avoir un suivi du code et un journal de travail pour suivre mes démarchent

- **Design UI/UX** : je me renseigne sur des principes d'UI et d'UX pour réaliser une interface user friendly. Je réalise par conséquent des maquettes haute-fidélité et des user story complètes
- **Architecture logicielle** : je choisis mon environnement et technologie en fonction des besoins.
- **Développement/Staging** : je mets en place un environnement de développement sain et un environnement de staging. Je me renseigne donc sur les méthodes à ma disposition pour mettre en staging l'application web et les mets en pratique.

1.2.2 Objectifs du produit.

Les objectifs de produit sont :

- **Authentification** : Prallo contient 3 rôles : la Gouvernance ou Doyen, Professeur et étudiant. Pour se faire un moyen d'authentification nous permet de savoir qui a quel rôle.
- **Annoncer les compétences** : les professeurs sont capables d'enregistrer leurs technologies favorites.
- **Nombre de projets** : les Doyens assignent un nombre max de projet par professeurs
- **Liste de projets** : Les professeurs peuvent créer des projets. Ceux-ci sont visibles par les étudiants dans une liste quand l'état de l'application le permet.
- **Voter pour des projets** : les élèves votent pour des projets qui leur donnent envie quand l'état de l'application le permet
- **Gérer la répartition** : un algorithme de tri fait la répartition des projets à l'aide d'un indice de satisfaction.
- **Gérer l'état du système** : les doyens gèrent l'état du système. Il en a 4 : Préparation, Vote, Répartition et Publication

1.3 Gestion de projet

1.3.1 Outils

Pour la gestion de ce projet, les outils sont les suivants :

- **Journal de travail** : la réalisation d'un journal de travail sous format Excel permet de suivre mes démarches
- **Git Project** : permet de créer des User stories exhaustives dans un Kanban et aide pour le suivi du développement.
- **GitHub** : Permet de contrôler les versions avec des releases

1.3.2 Pratiques

Le projet est conduit selon les **méthodologies agiles**, privilégiant l'adaptation continue et la communication transparente. Dans ce cadre, des échanges réguliers avec le client sont essentiels pour absorber les nombreuses itérations possibles.

Comme mentionné précédemment, l'utilisation de **GitHub Projects** permet de :

- **Implémenter un tableau Kanban** : pour une visualisation en temps réel de l'avancement des tâches et une gestion des priorités.
- **Établir une "Definition of Done" (DoD)** : pour garantir que chaque fonctionnalité répond à des standards de qualité stricts (tests, revues de code) avant d'être considérée comme terminée.
- **Gérer le cycle de vie des User Stories** : chaque fonctionnalité est rédigée sous forme de User Story afin de se concentrer sur la compréhension entre le client et moi.

Ces **User Stories** servent de base de négociation et de compréhension commune. Elles font l'objet d'une **revue systématique avec le chef de projet et le client** avant leur intégration au backlog de développement. Ce processus de validation itératif permet de s'assurer que les spécifications définies correspondent parfaitement aux attentes métier avant de lancer les développements techniques.

1.3.3 Project-Handbook

Durant ce projet, le respect des règles disponibles dans le repository GitHub "[Project Handbook](#)" **fait foi**. Voici quelques règles notables :

L'analyse fonctionnelle est réalisée au travers d'une liste de User Stories (US), comportant :

- *Un titre*
- *Une description ("En tant que ... je veux ... pour ...")*
- *Un ou plusieurs éléments graphiques (maquette, schéma, diagramme,...)*
- *Un nombre significatif de tests d'acceptance*

Ou encore, concernant GitHub :

- *Créez un repo privé dans votre compte et invitez-moi en lecture seule.*
- *Suivez la convention de nommage des commits [conventional commits](#).*
- *Appliquez le modèle [gitflow](#) pour la gestion des branches, avec les branches main, develop et les branches de features.*
- *Lors des sprint reviews, les fonctionnalités sont évaluées dans leur branche de feature. Si la story est validée, la branche de feature est fermée.*
- *Tous les fichiers utiles au travail et autres que du code (documentation technique interne ou externe, rapports, exemples, snippets, etc...) se trouvent dans l'arborescence sous doc, dont la structure est livrée à votre bon sens.*
- *Bien que cela soit une pratique controversée, je demande à ce que les fichiers concernant la documentation (notamment le rapport et le journal de travail) soient également intégrés dans le repository, dans le dossier doc.*

1.3.4 Personnes impliquées et rôles

- **Chef de projet et client** : *Xavier Carrel*
- **Expert 1** : *Nicolas Borböen*
- **Expert 2** : *Nemanja Pantic*
- **Apprenti** : *Théo Minger*

1.4 Planification initiale

Ce projet est réalisé en deux temps : une première partie qui consiste en la rédaction du rapport et du projet et une deuxième consacrée à la présentation.

La première partie durera du 07.05.2026 jusqu'au lundi 01.06.2026.

Une seconde partie concerne une soutenance qui aura lieu entre **le mercredi 10.06.2026 et le jeudi 11.06.2026.**

L'horaire est le suivant :

Lundi	8h-11h25	12h20-15h
Mardi	8h-11h25	13h10-16h35
Mercredi	8h-11h25	12h20-15h
Jeudi	8h-12h25	13h10-16h35
Vendredi	8h-12h15	13h10-16h35

Semaine du lundi 04.05.2026 : 13 périodes

Semaine du lundi 11.05.2026: 22 périodes

Semaine du lundi 18.05.2026: 40 périodes

Semaine du lundi 25.05.2026: 33 périodes

Semaine du lundi 01.06.2026: 4 périodes

Total : 90h.

Dates importantes : **Ascension 14 mai, Pont de l'Ascension 15 mai, Pentecôte 25 mai**

La planification est la suivante :

#	Objectifs	Nombres d'heures	Revue
1	Introduction + introduction du Rapport	6	Vendredi 08.05
2	Réalisation des maquettes + User Story (Analyse)	25	Lundi 18.05
3	Ecriture de la partie 2. Analyse / Conception du Rapport	13	Mercredi 20.05
4	Réalisation du code + Partie 3. Réalisation du Rapport	28	Mercredi 27.05
5	Réalisation des tests E2E + Fonctionnels	5	Mercredi 27.05
6	Mise en Production	10	Vendredi 29.05
7	Conclusion	3	Lundi 01.06
	Total	90	

Dans le rapport il est indiqué :

Analyse 25%, Implémentation 45%, Tests 10%, Documentation 10%. Néanmoins en vue de la nature de ce projet, la partie analyse et documentation sont censées prendre plus de temps, le planning en pourcentage est donc le suivant :

Analyse	Implémentation	Test	Documentation
30%	30%	10%	30%

2 Analyse / Conception

2.1 Lexique

- **Un état** : Dans PRALLO, il est question d'état d'application. Un état d'application représente la situation ou la configuration de l'application à un instant donné (par exemple : Préparation, Publication). Il détermine les actions possibles pour les utilisateurs et l'affichage des données.
- **Une compétence/technologie** : Il s'agit d'un savoir-faire technique, d'un outil ou d'un langage de programmation maîtrisé par un utilisateur ou requis pour la réalisation d'un projet au sein de l'application (par exemple : React, Gestion de projet, Python, Méthodes Agiles).

- **Un participant/invité** : Un participant ou invité est une personne qui s'est connecté à l'application PRALLO. Ce participant est ensuite enregistré dans la base de données. Le seul droit des participant est de visualiser les projets dans l'état « Repérage ». Une fois connecté, les doyens attribuent le rôle souhaité aux invités
- **MSAL** (Microsoft Authentication Library) :
- **Services Azure** (Microsoft Entra ID) :

2.2 Contexte produit

Chaque année, le mois de février marque une étape importante pour les étudiants de quatrième année de l'ETML en informatique. En effet, c'est dans ces dates que les étudiants reviennent d'une période de stage en entreprise. Afin de préparer au mieux leur Travail Personnel de Fin d'Études (TPI), la filière organise un projet d'approfondissement, nommé P_APPRO. Il s'agit d'un module qui prépare les élèves au TPI.

Cependant, la mise en place de ces projets individuels représente un défi logistique majeur pour la direction. La filière doit composer manuellement avec beaucoup de variables complexes : le nombre de projets maximales par enseignant, les compétences techniques spécifiques des professeurs, l'obligation d'attribuer certains projets prioritaires, et le respect des préférences exprimées par les élèves.

Harmoniser ces critères à la main s'avère extrêmement embêtant et il peut y avoir des erreurs de temps en temps.

C'est dans ce contexte qu'intervient l'application **Prallo**.

Prallo permet de mettre en relation les doyens, les professeurs et les élèves pour décider des projets mis à disposition, et des projets que les élèves vont réaliser.

2.3 Contextes techniques

2.3.1 Opérationnel

L'application Prallo est un site web classique accessible uniquement par internet. Elle ne s'installe pas sur les ordinateurs des utilisateurs. Elle est développée spécifiquement pour être utilisée sur un ordinateur de bureau (desktop) et n'est pas optimisée pour les téléphones ou les tablettes (pas de responsive). Tout le monde se connecte au même site via son navigateur, ce qui permet de centraliser toutes les données au même endroit.

2.3.2 Staging

L'infrastructure est entièrement hébergée sur une machine virtuelle Ubuntu 24.04.2 LTS installée sur le serveur w3 de la filière informatique. Le système s'articule autour de trois composants clés :

- **Le Client (Terminaux utilisateurs)** : Les utilisateurs (étudiants, professeurs et membres de la gouvernance) accèdent à l'application directement depuis le navigateur web de leur ordinateur de bureau (*desktop*). Le produit est conçu pour une utilisation exclusive sur ordinateur.
- **Le Serveur d'Application** : Une instance unique Ubuntu 24.04.2 LTS centralise le Front-end (Vue.js) et l'API Back-end (AdonisJS). C'est elle qui porte la logique métier, exécute l'algorithme de répartition sous contraintes et gère les états du système.
- **MySQL** : Une base de données MySQL assure la persistance et le stockage de toutes les données du projet (les comptes utilisateurs, les compétences des professeurs, les énoncés de projets, etc ...)

2.3.3 Développement

L'environnement de développement est conçu pour être simple à installer. Le frontend Vue.js et le backend AdonisJS tournent directement sur la machine de développement. Seule la base de données MySQL est conteneurisée via Docker, ce qui évite toute installation locale lourde tout en restant totalement transparente pour le reste du projet.

- **Prérequis** : Un développeur reprenant le projet doit installer deux outils sur sa machine : Node.js (version 20 ou supérieure) et Docker Desktop. Le développeur doit installer les dépendances une fois dans le projet. Aucune autre installation de base de données n'est nécessaire.
- **Base de données** : Un fichier docker-compose.yml situé à la racine permet de monter instantanément une instance MySQL. La base de données tourne de manière isolée et persiste ses informations grâce aux volumes Docker. Le développeur peut initialiser la structure des tables (compétences, projets, rôles) et injecter des utilisateurs de test (Gouvernance, Profs, Étudiants fictifs avec les seeder disponible) en une seule commande de migration et d'alimentation
- **Variables d'environnement** : Un fichier .env.example est présent dans le dépôt Github. Il contient la structure nécessaire au projet. Les données sensibles sont transmises de manière sécurisée entre les partis.

2.4 L'architecture et les services en schéma

2.4.1 Schéma de l'environnement de développement

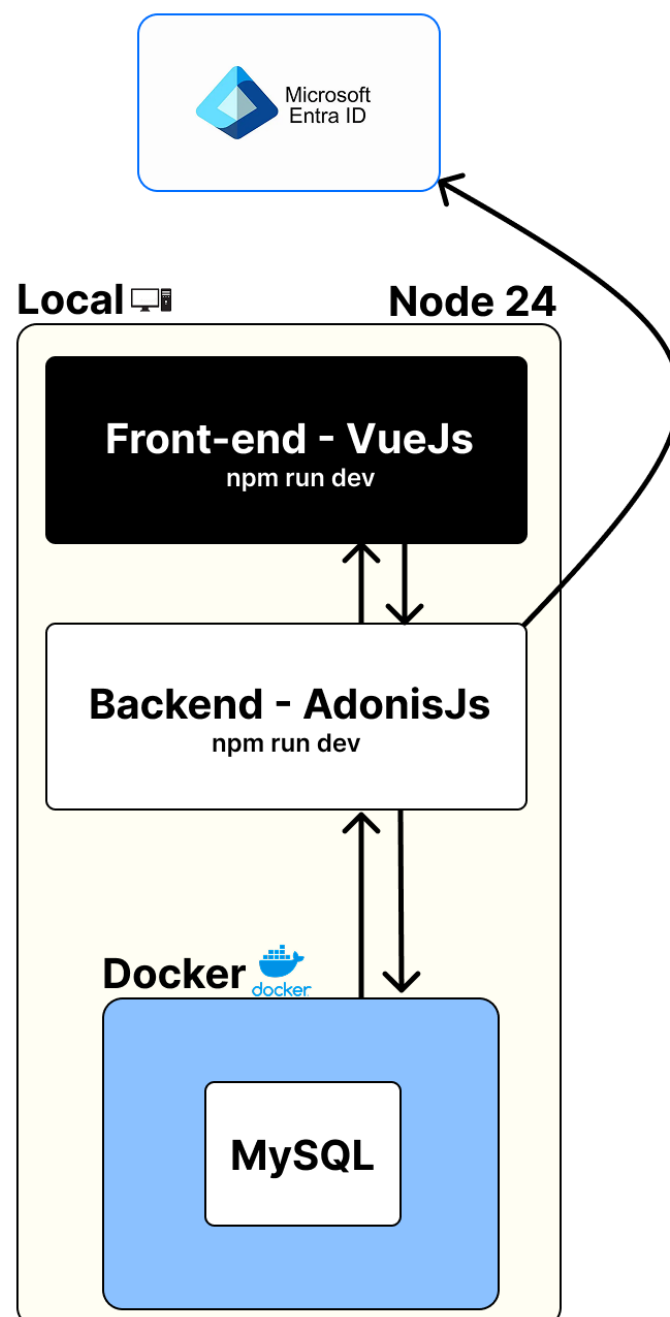


Figure 1 : Environnement de développement

2.4.2 Schéma de l'environnement de staging

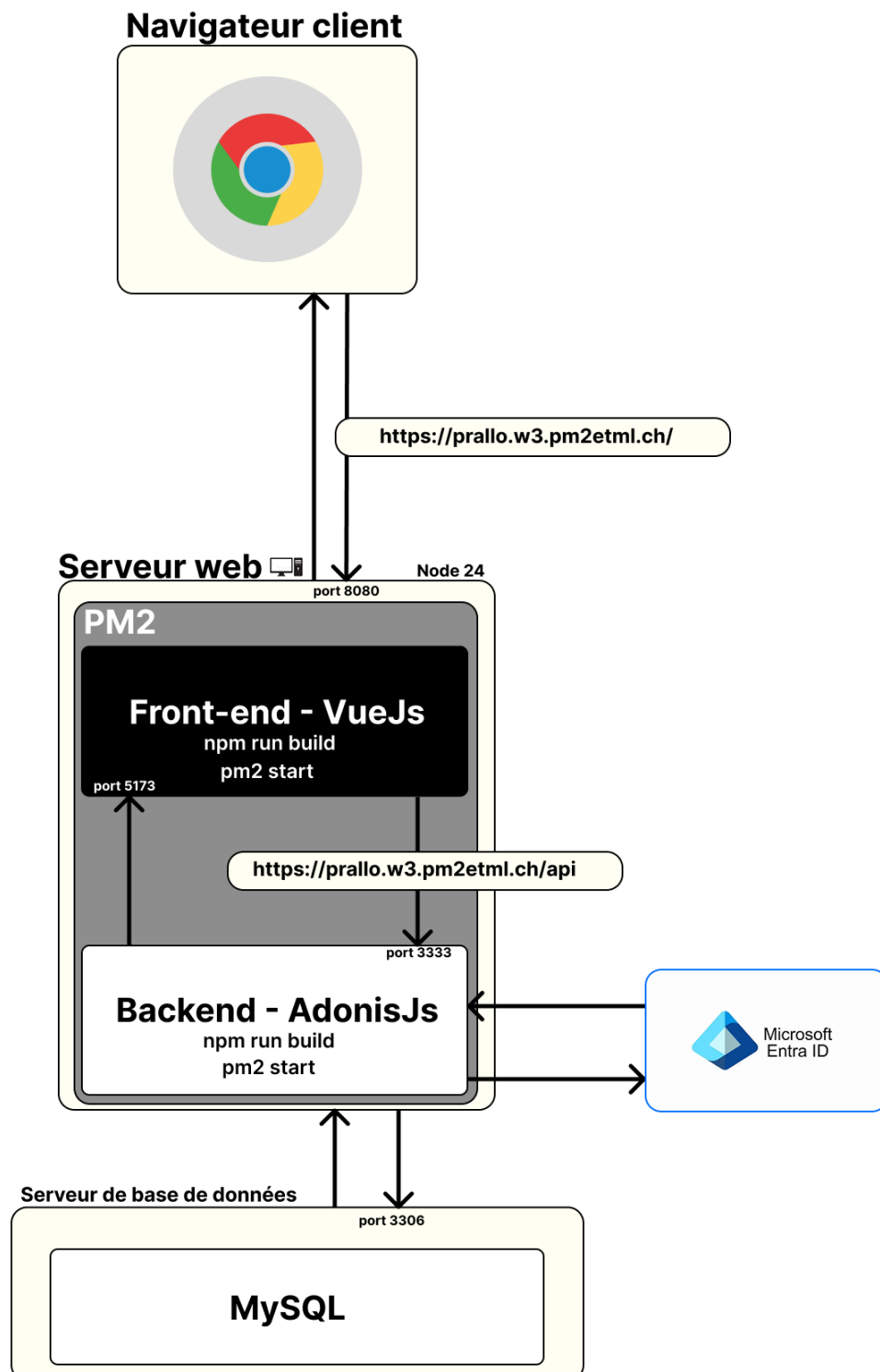


Figure 2 : environnement de staging

La disponibilité de l'application PRALLO est assurée par **PM2**. Cet outil sert de "gardien" pour le serveur web :

- **Gestion des processus** : Il fait tourner en arrière-plan le **Front-end (VueJS)** et le **Backend (AdonisJs)**.
- **Surveillance (Monitoring)** : Il permet également de surveiller en temps réel les performances du serveur, comme l'utilisation de la mémoire RAM et du processeur (CPU).

2.4.3 Diagramme d'état

Etat du système PRALLO

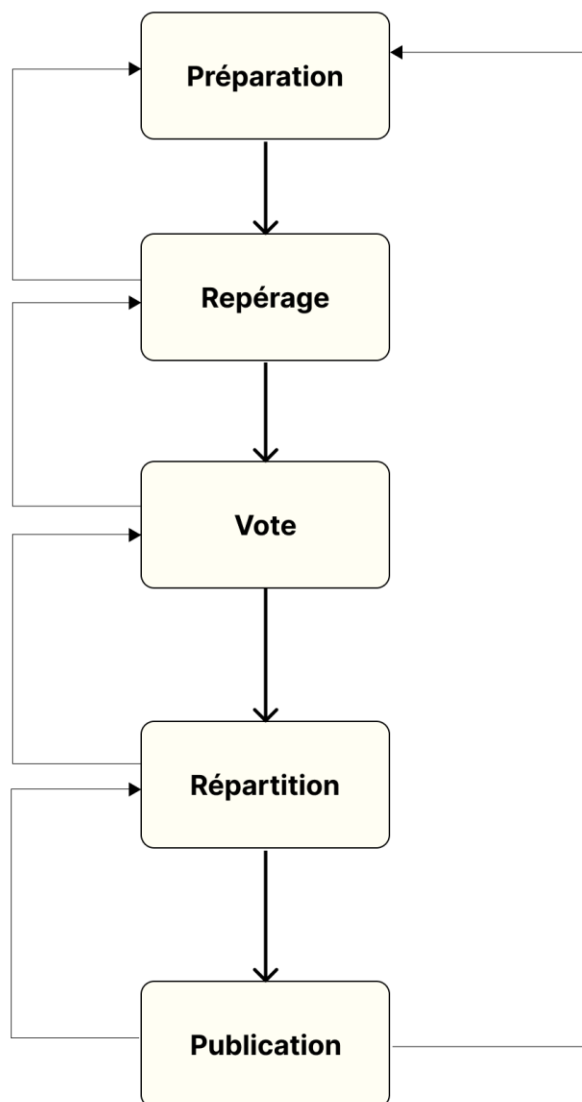


Figure 3 : Diagramme d'état

Le diagramme ci-dessus illustre les différents états par lesquels passe le système PRALLO (**Préparation, Repérage, Vote, Répartition, Publication**).

Le flux de l'application est conçu de manière flexible : **il est possible à tout moment de passer à l'état suivant ou de revenir à l'état précédent** afin de s'adapter aux réalités du terrain et aux corrections nécessaires. Il n'est juste pas possible de revenir en arrière une fois que le cycle a été fini (c'est-à-dire, il est impossible de passer de « Publication » à « Préparation » et de revenir en arrière)

Cependant, changer d'état de manière non linéaire ou prématurée peut impacter la cohérence des données (par exemple, revenir à l'état de *Vote* alors qu'une *Répartition* a déjà été amorcée). C'est pourquoi **l'application affiche des messages d'avertissement (*warnings*)** lors de ces transitions. Ces alertes informent l'utilisateur des problèmes potentiels ou des pertes de données possible avant qu'il ne confirme son action, garantissant ainsi la flexibilité du système sans compromettre sa sécurité.

Voici un exemple d'utilisation des états de PRALLO :

1. **Préparation** : la période de P_APPRO approche et il faut donc lancer PRALLO, un administrateur initialise le site web. Comme convenu, la première connexion crée un compte doyen. La personne en charge communique à la gouvernance que le service est prêt et qu'ils doivent se connecter. L'administrateur leurs attribue ensuite le rôle de doyen. Les doyens demandent aux professeurs de se connecter. Il se passe exactement la même chose, mais avec les professeurs. La gouvernance communique une date à laquelle le site va passer en état « Repérage ». En attendant les professeurs choisissent leurs compétences et ajoutent des projets.
2. **Repérage** : Dans cet état, rien ne change pour les doyens et les professeurs. Les élèves/invités peuvent venir se renseigner sur les projets disponibles.
3. **Vote** : Les professeurs ne peuvent plus supprimer de projet, mais ils peuvent les modifier. Les élèves votent pour leurs projets préférés jusqu'à une date butoir.
4. **Répartition** : La répartition des projets est visible par tout le monde, seuls les doyens peuvent encore modifier les projets.
5. **Publication** : plus personne ne peut rien faire, hors-mi visualiser la répartition

Il est important de notifier que aucune condition pour passer d'un état à un autre n'est demandé

2.4.4 Diagramme de flux concernant l'authentification

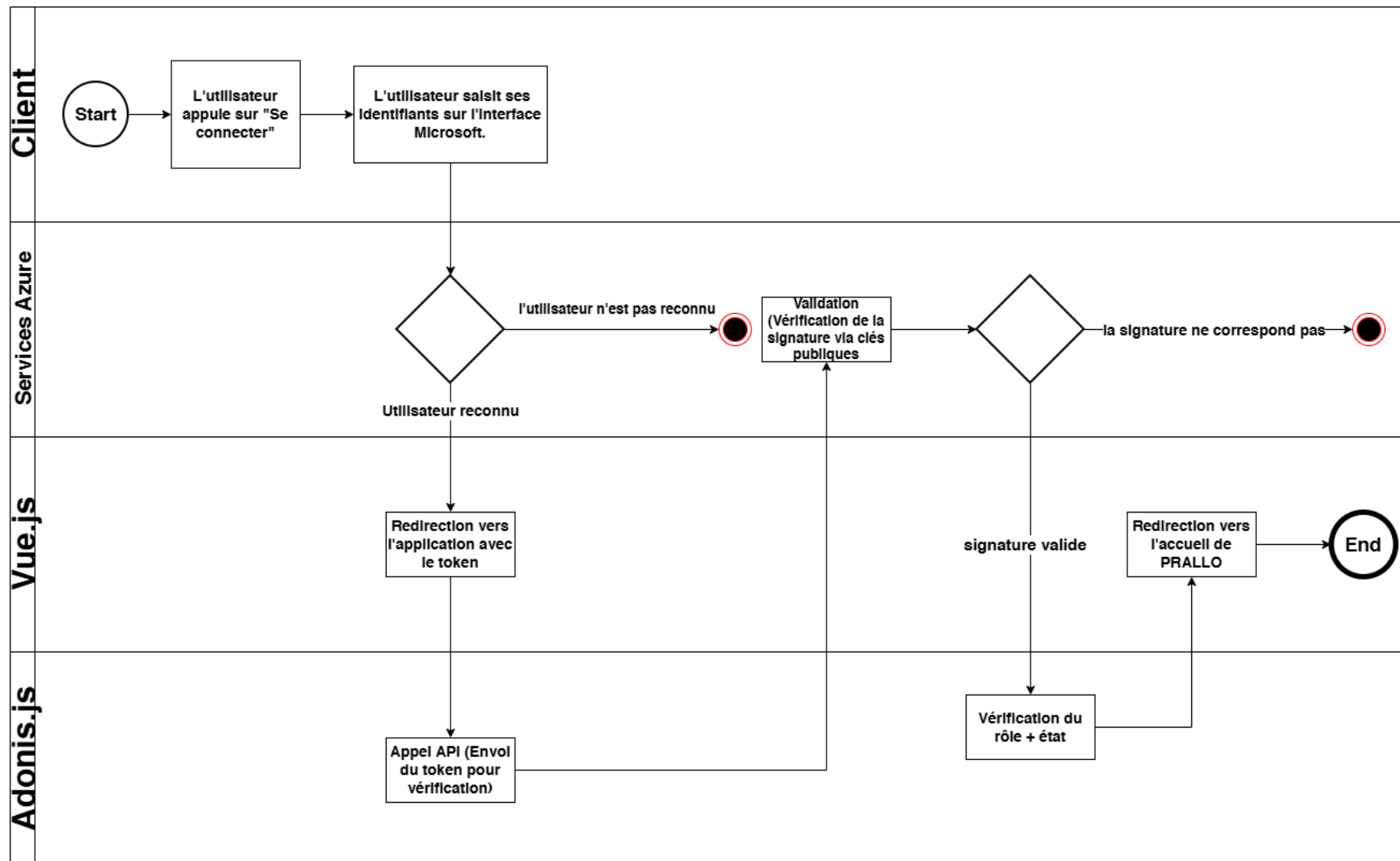


Figure 4 : diagramme de flux

Le diagramme d'activité ci-dessus détaille le processus d'authentification sécurisé et le routage dynamique mis en place au sein de l'application PRALLO. Ce flux s'appuie sur la bibliothèque **MSAL** (Microsoft Authentication Library) intégrée au Frontend et sur les **Services Azure** (Microsoft Entra ID) comme fournisseur d'identité.

Le processus se déroule selon les étapes suivantes :

1. **Phase d'authentification (Client & Azure)** : L'utilisateur initie la connexion depuis l'interface client. Il est alors invité à saisir ses identifiants directement sur l'interface sécurisée de Microsoft. Si l'utilisateur n'est pas reconnu par Azure, le processus s'interrompt immédiatement.
2. **Émission du Token (Azure & Vue.js)** : Une fois l'utilisateur authentifié avec succès, les Services Azure opèrent une redirection vers le Frontend **Vue.js** en lui transmettant un jeton d'accès sécurisé (Token).
3. **Vérification de sécurité (Adonis.js & Azure)** : Le Frontend transmet immédiatement ce jeton au Backend **Adonis.js** via un appel API d'authentification. Pour garantir l'intégrité du jeton, le Backend interroge les Services Azure afin de valider mathématiquement la signature via leurs clés publiques. Si la signature s'avère invalide ou altérée, la connexion est immédiatement rejetée.
4. **Contrôle d'accès et routage dynamique (Adonis.js & Vue.js)** : Dès que la signature est validée, le Backend exécute un middleware chargé de récupérer à la fois les permissions en fonction de son rôle et l'**état actuel du système PRALLO** (ex: Préparation, Vote, Répartition, Publication). C'est l'évaluation conjointe de ces deux variables (Rôle + État) qui dicte au routeur **Vue.js** vers quelle page d'accueil spécifique l'utilisateur doit être redirigé et quelles actions il est capable de faire.

2.5 Analyse fonctionnelle

2.5.1 Discussion avec le Client/Product Owner

Lors de l'analyse du cahier des charges, il est clair que des points restent à éclaircir. C'est pourquoi dans la période d'analyse, j'ai pris du temps pour avoir des rendez-vous avec le product owner/client (Ici, le chef de projet, Monsieur Carrel, représente les 2 rôles en même temps).

Lors de ces discussions, j'ai posé plusieurs questions et **des changements dans le cahier des charges ont eu lieu** :

- **Authentification** : un point important de l'application PRALLO est l'authentification et l'attributions des rôles. Il était question de comment initialiser l'application, c'est-à-dire : Comme ce sont les doyens qui attribuent les rôles, comment fait-on pour créer le premier profil doyen ? passons-nous par la base donnée ? Avons-nous une liste de doyen que nous initialisons ? La réponse à cette question fut trouvée : **La première personne qui se connecte au service PRALLO, est enregistrée avec le rôle « Doyen »**, Grâce à ce système, l'administrateur qui initialisera le service, pourra changer les rôles des autres participants en doyen lors de leurs connexions.
- **Attribution des rôles** : Dans le paragraphe d'au-dessus, il est question de « changer le rôle des autres participants » mais c'est-à-dire ? Dans PRALLO, les doyens auront le droit d'attribuer des rôles au participants. **La gouvernance ne peut donc pas éditer librement la liste des participants, contrairement à ce que laisse penser le cahier des charges**. Elle ne peut pas supprimer de profils ni en créer, mais elle a le droit de modifier par exemple le rôle des invités
- **Rôle + Etat** : En ayant lu les paragraphes d'au-dessus, vous avez peut-être remarqué que nous parlons d'un rôle participant/invité et d'un état « Repérage ». Ceux-ci ne sont pas disponible dans le cahier des charges, **ce sont des ajouts au cahier des charges**, après discussion avec le Product Owner.

Les changements ne sont ici pas tous détaillés, les user story sont là pour définir les fonctionnalités disponibles dans l'application PRALLO.

2.5.2 User story

L'analyse fonctionnelle est constituée de toutes les user story réalisé dans le GitProject
Une User story est écrite de la manière suivante :

- **Un titre** : Il s'agit de la fonctionnalité que l'on souhaite développer.
- **Une description** : C'est ici que l'on indique qui veut pouvoir faire l'action, quelle action cette personne voudrais faire et pour quel but. Le formalisme est le suivant : *En tant que ... je veux ... afin de ...*
- **Les critères d'acceptation** : C'est ici que la User Story prend vie. En associant des maquettes visuelles à des règles de gestion précises, cette partie définit exactement le comportement de l'application. L'enjeu est d'être suffisamment minutieux pour que n'importe quel membre de l'équipe (développeur, testeur ou client) puisse se projeter dans l'application et en saisir immédiatement les moindres détails.

- **Gestion d'erreur** : Cette section liste les points de vigilance techniques et les erreurs potentielles à anticiper lors du développement (par exemple, la limite maximale de caractères pour un champ de saisie). Ces précisions sont indispensables, car elles servent de base directe à la conception des scénarios de test, conformément aux exigences de la Definition of Done (DoD) et de la stratégie de test globale du projet.
- **De labels** : L'application Prallo intégrant plusieurs rôles, il est essentiel de pouvoir s'y retrouver facilement dans le GitProject. Pour cela, l'utilisation des labels afin d'attribuer une étiquette précise à chaque tâche selon le profil concerné (par exemple : « Doyen » ou « Professeur »), permet de filtrer rapidement le tableau et de clarifier le suivi.

Les US se retrouvent en annexe.

2.6 Stratégie de test

Chaque niveau de test joue un rôle spécifique et indispensable pour s'assurer que le code livré répond aux attentes en termes de fiabilité, de performance et de convivialité.

1. Tests d'acceptance (Critères d'acceptance)

Les tests d'acceptance valident que le système répond exactement aux exigences métier définies dans les User Stories. Ils s'exécutent dans un contexte au plus proche de l'environnement de production et simulent un flux utilisateur complet.

- **Rôle** : Ils définissent clairement ce qu'un développement doit accomplir avant d'être considéré comme terminé. Un test réussi signifie que la fonctionnalité répond aux critères convenus avec les métiers.
- **Anticipation** : Ces cas de test sont documentés dans les User Story afin de valider les comportements attendus de bout en bout.

2. Tests unitaires (Jappa et Vitest)

Les tests unitaires permettent de vérifier la logique métier et les cas limites. Chaque composant est isolé et testé individuellement pour s'assurer qu'il gère correctement les erreurs.

Pour cette catégorie, le projet s'appuie sur deux outils distincts :

- **Jappa (API REST)** : Utilisé pour tester la logique du back-end, la validation des routes et la gestion des cas exceptionnels.
- **Vitest (Code Vue)** : Un framework moderne et rapide, intégré nativement à l'écosystème Front-end qui est en Vue.js

Il est convenu dans la DoD que les points sous la rubrique « Gestion d'erreur » font le point de test unitaire

3. Tests End-to-End (Cypress)

Les tests End-to-End (E2E) avec Cypress visent à vérifier l'intégration globale du système en simulant des scénarios réels, de l'authentification initiale jusqu'à la validation finale d'une action.

- **Validation visuelle et fonctionnelle** : Cypress parcourt l'interface comme un utilisateur réel, On s'assure qu'au moins un élément visuel clé s'affiche correctement sur chaque page du DOM.
- **Suivi et robustesse** : Ce framework garantit que l'expérience utilisateur reste cohérente, fluide et sans régression. De plus, il offre une excellente visibilité sur l'exécution des tests grâce à son interface interactive et ses replays vidéo.

Il est convenu dans la DoD que au minimum un élément visuel sois testé par vue.

2.7 Risques techniques

- **Disponibilité des systèmes externes** : L'application dépend entièrement de Microsoft Entra ID. Si le service Azure subit une panne ou une indisponibilité, le système d'authentification devient inaccessible et plus personne ne peut se connecter à PRALLO.
Solution : On pourrait imaginer un deuxième moyen de connexion si le premier tomberait en panne. Néanmoins ce projet ne contiendra qu'un moyen de connexion
- **Complexité de l'authentification MSAL** : Configurer les redirections entre le local, le staging et la prod est complexe.
Solution : être au clair avec le système MSAL et centralisé les données possibles dans le fichier .env
- **Complexité au niveau des fonctionnalités** : Vu l'exhaustivité des user story, Il existe un risque réel de passer à côté de certaines fonctionnalités secondaires ou de mal interpréter des cas limites lors du développement.
Solution : Utiliser un tableau de suivi (type Jira ou Trello) pour cocher rigoureusement chaque User Story.
- **Dépendance au résultat de l'intelligence artificielle pour rester dans les délais de livraison** : Comme la réalisation du code est fait entièrement avec l'aide de l'IA, si celui-ci prend beaucoup plus de temps qu'imaginé, que le code demande du débogage long, le délai de livraison pourrait en être impacté.
Solution : Planifier des code review à l'ajout de fonctionnalités importantes

2.8 Planification

Un tableau permet de visualiser la date à laquelle on aimerait avoir fini les fonctionnalités. Ces dates feront l'objet de « Sprint review » dans la partie 3. Réalisation. Si des retards ou des problèmes devraient avoir lieu, c'est plus tard que le rapport en discutera.

Titre de l'user story	Rôle impliqué	Date prévu
S'authentifier	Utilisateur	26.05.26
1ère authentification	Administrateur	26.05.26
Gérer l'état du système	Doyen	26.05.26
Afficher la liste des participants	Doyen	26.05.26
Visualiser les projets	Doyen	26.05.26
Visualiser les projets	Professeur	26.05.26
Visualiser les projets	Élève	26.05.26
Visualiser les projets	Invité	26.05.26
Suppression de projet	Doyen	26.05.26
Modifier des projets	Doyen	26.05.26
Visualiser mes compétences	Professeur	26.05.26
Visualiser les compétences de PRALLO	Professeur	26.05.26
Ajouter des compétences	Professeur	26.05.26
Supprimer des compétences de ma liste	Professeur	26.05.26
Changer le statut des participants	Doyen	26.05.26
Attribuer un nombre de projet	Doyen	26.05.26
Ajouter des projets	Professeur	28.05.26
Supprimer mes projets	Professeur	28.05.26
Modifier mes projets	Professeur	28.05.26
Voter pour un projet	Élève	28.05.26
Supprimer mon vote d'un projet	Élève	28.05.26
Visualiser les projets votés	Élève	28.05.26
Mettre dans l'ordre de préférences	Élève	28.05.26
Visualiser les compétences	Doyen	28.05.26
Supprimer les compétences	Doyen	28.05.26
Fusionner des compétences	Doyen	28.05.26
Ajouter des compétences à ma liste	Professeur	28.05.26
Faire la répartition	Gouvernance	28.05.26

3 Réalisation

3.1 Déroulement

3.2 Sprints

3.2.1 Fin du premier Sprint, date 26.05.2026

A la fin de journée 26.05, si on regarde le tableau suivant :

S'authentifier	Utilisateur	26.05.26
1ère authentification	Administrateur	26.05.26
Gérer l'état du système	Doyen	26.05.26
Afficher la liste des participants	Doyen	26.05.26
Visualiser les projets	Doyen	26.05.26
Visualiser les projets	Professeur	26.05.26
Visualiser les projets	Élève	26.05.26
Visualiser les projets	Invité	26.05.26
Suppression de projet	Doyen	26.05.26
Modifier des projets	Doyen	26.05.26
Visualiser mes compétences	Professeur	26.05.26
Visualiser les compétences de PRALLO	Professeur	26.05.26
Ajouter des compétences	Professeur	26.05.26
Supprimer des compétences de ma liste	Professeur	26.05.26
Changer le statut des participants	Doyen	26.05.26
Attribuer un nombre de projet	Doyen	26.05.26

Toutes les fonctionnalités hors-mi l'authentification sont « presque » finis et disponibles dans le kanban git Project sous la rebrique in-review. C'est à ce moment là qu'entre en jeu ma Définition of Done. Si la user story se trouve dans la rebrique in-review, elle doit passer les critères de la DoD pour finir en « Done ». Pai ailleurs ma DoD est la suivante :

- Si la tâche contient des commentaire sous # Gestion d'erreur, celle(s)-ci doivent être contrôlé par des tests unitaires/fonctionnelles
- La US a été validé par le Product Owner
- Le front-end a été validée par l'équipe de développement
- Au moins 1 élément visuel a été testé via un test E2E

Elle est aussi disponible dans la description du gitProject.

Durant ce sprint, j'ai :

- **Organisation et Workflow :** J'ai structuré le travail autour du modèle **Gitflow** pour garder une base de code propre et organisée. Aujourd'hui, tout ce qui a été développé est centralisé dans le Kanban de GitProject, dans la colonne *In Review*. En gros, le gros du travail est fait, il ne reste plus qu'à passer chaque fonctionnalité au crible de notre *Definition of Done* pour les valider officiellement.

- **Méthodologie "Prompt Bullet"** : Pour gagner en efficacité, j'ai testé une approche itérative en m'appuyant sur l'IA. J'ai conçu un "prompt maître" au format Markdown qui intégrait toute notre documentation technique, ce qui m'a permis de générer une première version solide des fonctionnalités très rapidement. Bien sûr, cette base "full IA" n'était qu'une étape : j'ai ensuite repris chaque partie pour effectuer une revue technique détaillée, m'assurer que le code était cohérent et qu'il correspondait exactement à nos besoins.
- **Fiabilisation et Correctifs** : Une fois la structure en place, j'ai passé pas mal de temps à "nettoyer" et ajuster le résultat. J'ai corrigé les bugs qui apparaissaient lors de mes tests et j'ai ajouté les fonctionnalités qui manquaient ou qui méritaient quelques finitions sur la logique métier. L'objectif était vraiment de transformer cette base générée en un produit qui correspond au cahier des charges

3.2.2 Stories

3.3 Description des tests effectués

3.4 Bilan

3.4.1 Erreurs restantes

3.4.2 Stories

3.4.3 Dette technique

3.5 Recours à l'intelligence artificielle

3.6 Liste des documents fournis

4 Conclusions

5 Annexes

5.1 Résumé du rapport du TPI / version succincte de la documentation

5.2 Sources – Bibliographie

5.3 Journal de travail

5.4 Manuel d'Utilisation